

Term	Definition	Example
Entity Type	A collection of entities with the same properties.	Student entity type with studentID, Name, Email.
Entity Set	All entities of a given entity type at a specific time.	All students enrolled in the university.
Intension of an Entity	Schema definition (structure) of an entity type.	Student(studentID, Name, Email).
Extension of an Entity	Actual data/tuples stored in the entity set.	{ (1, 'Alice', 'a@uni.edu'), (2, 'Bob', 'b@uni.edu') }.
Attribute	A property of an entity or relationship.	Name, studentID, Age.
Domain of an Attribute	Possible values an attribute can take.	Age -> Integers from 15 to 100.
Null Value	A missing, unknown, or inapplicable attribute value.	Phone = NULL (if student didn't provide it).
Super Key	Any set of attributes that uniquely identify an entity.	{studentID}, {Email, Name}.

Candidate Key	A minimal super key (no redundancy).	studentID (uniquely identifies a student).
Primary Key	The chosen candidate key for unique identification.	studentID = Primary Key.
Alternate Key	Candidate keys not chosen as primary.	Email (if RollNo is primary).
Relationship Type	A description of a relationship between entity types.	Enrolled (Student–Course).
Relationship Set	All instances of a relationship type.	{ (Alice, Math101), (Bob, CS201) }.
Binary Relationship	A relationship between two entity types.	Student–Enrolled–Course.
Ternary Relationship	A relationship among three entity types.	Supplier–Supplies–Product–Project.
N-ary Relationship	A relationship among n entity types.	Doctor–Patient–Hospital–Treatment.
Cardinality of Relationship	The number of entity instances that can be related.	One-to-Many: Student–Enrolled–Course.

Weak Entity	n entity that depends on another entity for identification	Dependent (weak) depends on Employee (strong).
-------------	--	--

2. When is the concept of a weak entity used in data modeling? Define the terms owner entity type, weak entity type, identifying relationship type, and partial key.

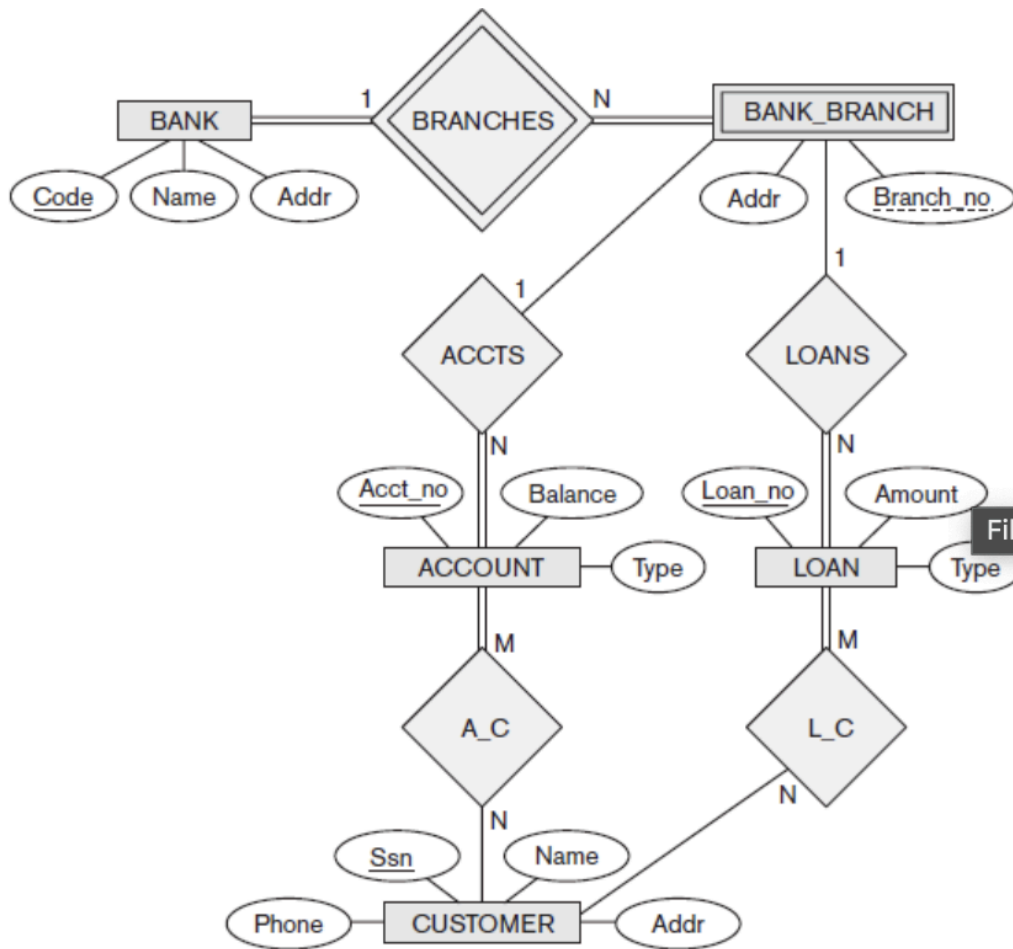
The concept of a weak entity is used when an entity cannot be uniquely identified by its own attributes alone and depends on another entity for identification and existence.

Owner entity type: The strong entity that provides the primary key used to help identify the weak entity. Example: Student(StudentID, Name, Major). StudentID is the primary key, and a student exists independently.

Weak entity type: An entity that cannot be uniquely identified by its own attributes. It needs the primary key of the owner entity plus its own partial key to form a unique identifier. Example: Dependent(Name, Age, Relationship). The attribute Name alone is not unique, so it requires StudentID from the owner entity.

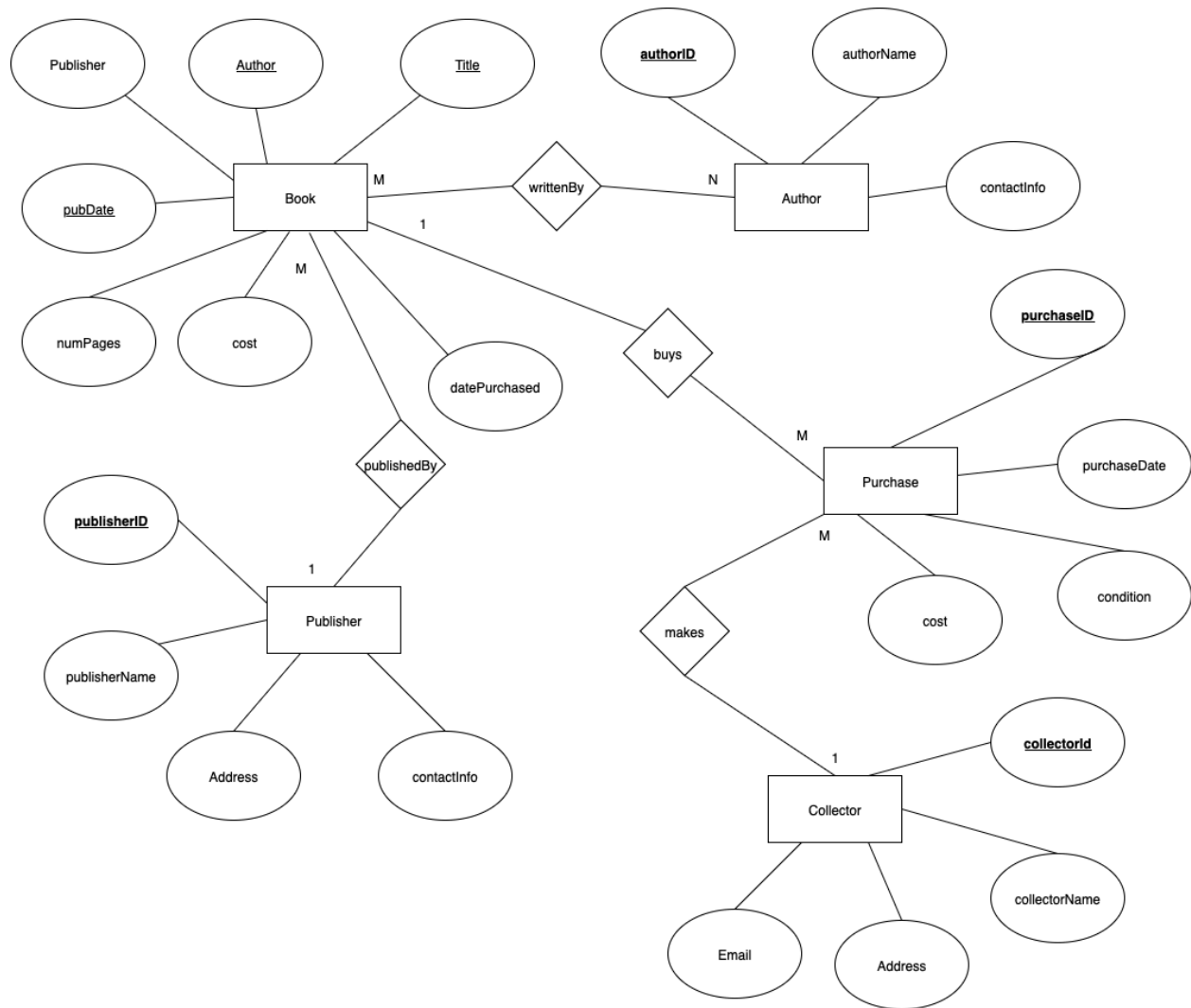
Identifying relationship type: The special relationship that links a weak entity to its owner. Example: HasDependent connects Student to Dependent. Without this relationship, a dependent cannot be uniquely identified.

Partial key: A set of attributes in the weak entity that can uniquely identify it within the context of the owner entity. Combined with the owner's primary key, it becomes the weak entity's full primary key. Example: Dependent(Name) is the partial key, so the full key is (StudentID + Name).



File Preview

3
A.



b) Domain of cost attribute:

Real numbers between 0 and 999.99

Cannot be negative since it is a cost of book

Example: DECIMAL (5,2) allows up to 999.99. For eg: 0.00 (free of cost), 7.45, 8.99, 12.05, 121.43, 567.89.

Currency can be USD (assumed) or depends upon the region.

c) A superkey is any attribute (or combination) that uniquely identifies a tuple.

Possible superkey:

1. {title, author, publisher, pubDate, datePurchased}
because two copies of the same book with same title/author/publisher/pubDate may exist, but *datePurchased* will distinguish them)

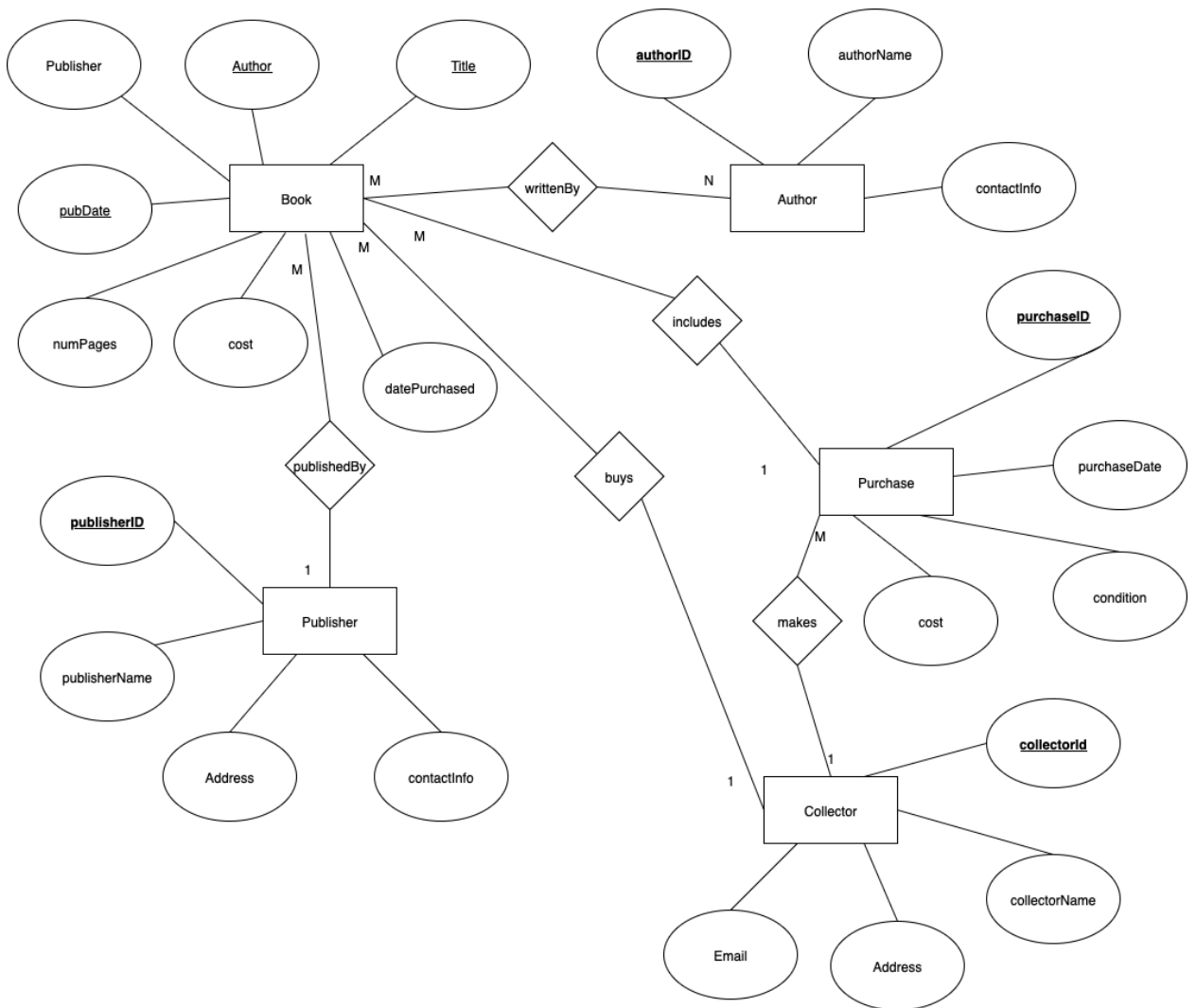
d) A candidate key is a minimal superkey (no attribute can be removed without losing uniqueness).

Candidate keys could be:

1. {title, author, publisher, pubDate, datePurchased}
 - Minimal and unique enough.
2. If a collector assigns a unique accession number or BookID (not in original schema but often realistic), that would also be a candidate key.

e) done in ER diagram. I chose a combination of attributes for BOOK entity since we did not have a subtle primary key. The other entities have collectorID, purchaseID, etc which are easy to locate and underlined in the ER diagram as well.

4. a)



I added “includes” relationship for Purchase entity with Book assuming every purchase includes a book.

I added a purchaseID as primary key for Purchase otherwise, purchaseDate alone isn't guaranteed unique.

The relationship is many-to-many at first glance, but since each book belongs to *exactly one purchase*, it resolves to N:1 (Book -> Purchase).

If we assume reselling/re-purchasing of books, then the relationship could instead be M:N with an associative entity like PurchaseDetail.

b)

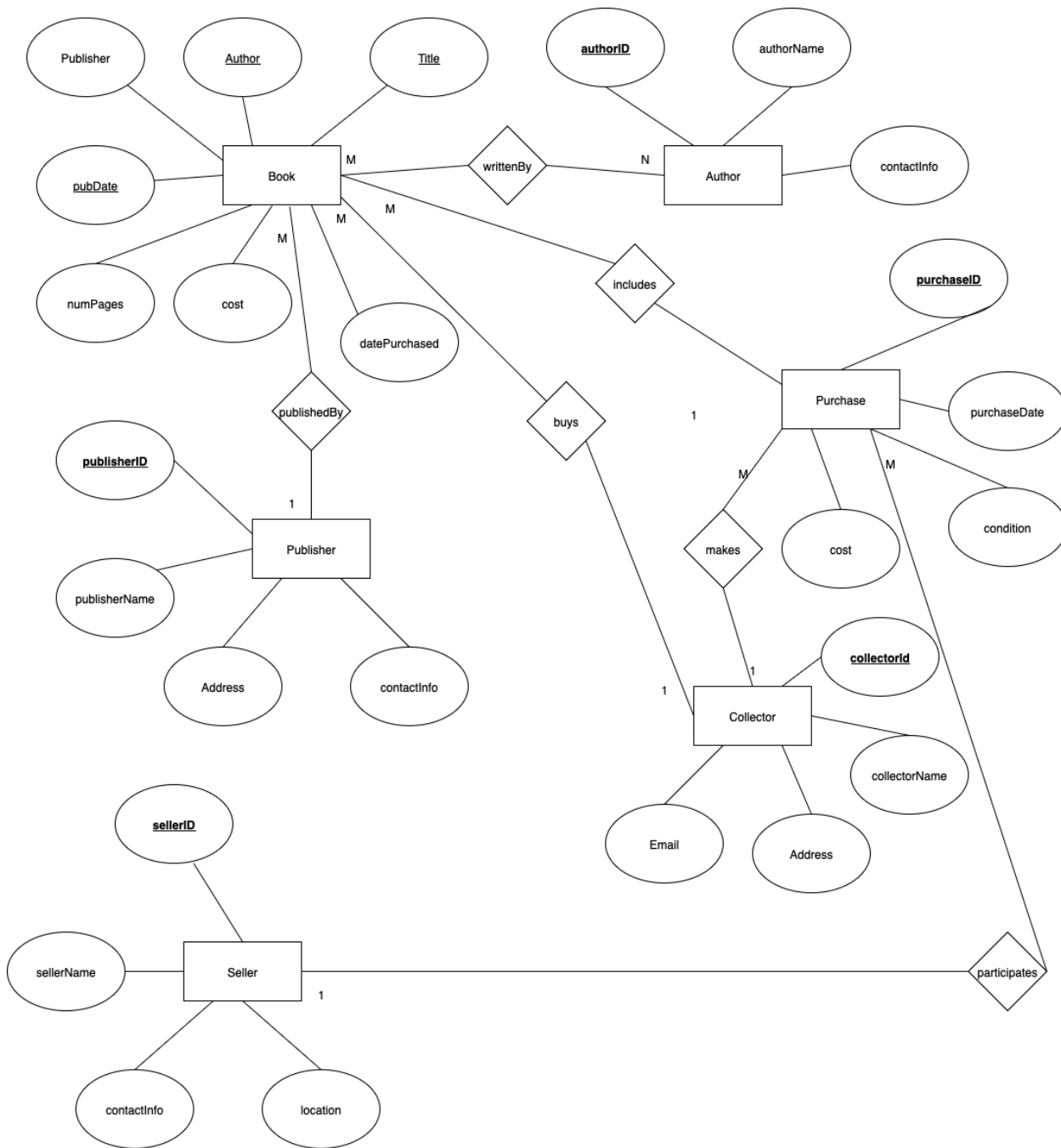
Every book in the collector's database must have been bought in a purchase. The book participates totally in Includes.

A purchase may include one or more books. Purchase has total participation too, since an empty purchase wouldn't make sense.

A purchase can contain many books, but each book is linked to exactly one purchase event (since the collector records purchase history).

Purchase and Book are 1-to-Many relationship as you can buy more than one book in a single purchase. The relationship is shown in the ER diagram above.

c)



Purchase (Includes)

- One purchase -> many books
- Each book -> exactly one purchase
- Total participation on both sides

Seller (participates)

- One seller -> many purchases
- Each purchase -> exactly one seller
- Purchase has total participation, Seller has partial participation